

Reliability of Task Execution during Safe Software Processing

Peter Raab, Stefan Krämer, Jürgen Mottok
Faculty of Electronics and Information Technology
Regensburg University of Applied Sciences
Seybothstr. 2, D-93053 Regensburg, Germany
 {peter.raab, stefan.kraemer, juergen.mottok}@hs-regensburg.de

Stansislav Raček
Faculty of Applied Sciences
University of West Bohemia
Univerzitní 22, 306 14 Plzeň, Czech Republic
 stracek@kiv.zcu.cz

Abstract—This paper presents the reliability evaluation of task execution during safe software processing. The standard method of duplication in a safety-critical application can also be applied for tasks in a software system. But in addition to this, there is also the possibility for coded task processing to increase the reliability and availability of software. The presented analysis covers the reliability analysis of a single, a duplicated and a coded task by the technique of continuous-time Markov processes. Markov processes are often used for the reliability evaluation of safety-critical systems. We introduce a method to describe the execution time of tasks by means of enhanced Markov models and their solution by numerical methods.

Keywords: reliability analysis, continuous-time Markov process, error probability, Erlang-distribution

I. INTRODUCTION

Fault-tolerant systems have become more important in recent years. Either for economic reasons or because of safety aspects, fault-tolerant systems are required to reduce costs or save life. Achieving this aim, the correct operation of the system has to be assured and failures must be detected and repaired.

The complexity and functionality of electronic control units have increased more and more in several sectors of industry. In addition, the requirements of these systems have become more demanding in terms of safety, reliability and availability. In contrast to this progress, industry demands a decrease in costs for electronics, while at the same time remaining competitive. The use of inexpensive commodity hardware is the result. However, the development of current micro-controllers follows the trend of decreasing feature size. That leads to less reliability and arbitrary hardware faults are more likely [1]. But despite unreliable hardware, fault tolerance is a requirement of safety-critical applications. This can often be realized by software techniques in many ways [2].

For this reason, current standards and norms for functional safety summarize several state-of-the-art techniques to detect possible errors in safety-critical systems. The European standard IEC61508 - "Functional Safety of E/E/P Safety-Related Systems" [3] demands for high safety-critical applications (SIL 3 to SIL4) a maximum rate from $3 \cdot 10^{-7}$ to $3 \cdot 10^{-9}$ per

hour of dangerous failures in the system. The norm describes the technique of coded processing and reciprocal comparison for detection of faults within the processing unit with high probability.

Today, there are a variety of publications about fault-tolerant computer systems. In [2], the authors present different state-of-the-art techniques of so-called Software-Implemented-Hardware-Fault-Tolerance (= SIHFT). They describe a simple possibility of hardening data against Single Event Upsets (= SEU) by duplication (= data redundancy) and multiple computation of data (= time redundancy).

But only transient faults can be detected by pure data and time redundancy. Permanent faults in the CPU (e.g. a stuck-at fault in the adder hardware) will generate the same erroneous result when an instruction is executed twice with the same data. The consequence is the usage of redundant hardware or of diverse data in the way that different units in a CPU are employed [4].

This paper is organized as follows. Sec. II gives an overview of the background and definitions for fault tolerance, coded software processing and reliability. The main part of this paper is Sec. III. In that section the reliability of a task in a software system is analyzed. Finally, Sec. IV summarizes the results of the analysis for further work in this research area.

II. BACKGROUND

This section gives an overview of the necessary background and state-of-the-art for the reliability analysis of coded task processing.

A. Reliability and the Markov Model

Reliability is a metric to describe the ability that a system has to perform the required function correctly for a specified period of time [5]. Stochastic Markov processes are a powerful technique that is often used for the reliability analysis of a given system (see [6] for detailed description of the technique). The basic concept of Markov processes is the partition of the system into several failure states. Each state identifies a certain fault condition of the system. Thus, the Markov process can be modeled as a kind of finite automata

consisting of nodes and transitions between them. The transition between two states represents a random event of a fault and its frequency is described by the transition rate. This rate can be interpreted as the reciprocal mean time of one transition. The underlying probability distribution of Markov processes is the exponential distribution with the transition rate as a parameter. Therefore, Markov processes are suitable for events that happen with exponentially distributed time, like hardware faults. More regular events like repairs or the runtime of certain software tasks follow another distribution (e.g. normal distribution) and the Markov process can not be used in its original form.

In order to evaluate tasks with fixed execution time, the Markov model must be enhanced to approximate other distributions by means of the exponential distribution. The *Erlang-distribution* [7], [8], [9] is derived by the consecutive stages of exponentially distributed transitions. The enhancement of a Markov process by adding further states realizes the Erlang-distribution and we can approximate other distributions like the normal distribution [10]. The evaluation in Section III makes use of this kind of Markov model to simulate the execution time of a single task.

B. Coded Software Processing

Coded Processing is the protection of calculations and their results during operations in an arithmetic unit by means of error detection codes. Channel coding, as a part of coding theory, describes error detection and correction codes like the Hamming code, which are originally developed for protecting stored or transmitted data. Another important group of error detecting codes are so-called *arithmetic codes* (AN-code) that are based on ordinary algebra like addition and multiplication. Forin made the first use of coded processing in a real application [11]. He defined coded operations for most arithmetic operations and extended signatures to this kind of code to detect operation, operator and operand errors. A detailed description of arithmetic codes for coded processing can be found in [12].

Since fault-tolerant computer systems have become more important in safety-critical applications, several institutes research this topic using error detecting codes for arithmetic operations. The Laboratory for Safe and Secure Systems at the University of Applied Sciences Regensburg developed, in collaboration with the TU Munich, the Safely Embedded Software (= SES) technique for the programming language C to safeguard the execution of code on microprocessors [13], [14]. Based on SES, an enhanced approach of safe software processing using concurrent task execution is presented in [15]. The basic idea of this approach is that the operating system generates two instances of the same task, which are executed in parallel. This time redundancy in combination with diverse coded data in each instance allows the detection of data errors by comparing both instances at certain points of time, so-called synchronization points. In addition, it

is also possible to compare the program counter of both instances. Thus, program flow monitoring is realized and deviations in the program flow are detectable.

III. RELIABILITY EVALUATION OF TASK PROCESSING

A linear program executes several operations consecutively and the data constantly changes. With a constant error rate, the result of an operation is less confident the longer the operation lasts. Usually, the operations of a task depend on each other. The result of one operation is the input for the next. The reliability of a complete task $R(t_{Task})$ has a serial structure and is the product of the reliabilities of the single operations R_1 and R_2 .

$$\begin{aligned} R(t_{Task}) &= R_1 \cdot R_2 = e^{-\lambda \cdot t_{op1}} \cdot e^{-\lambda \cdot t_{op2}} \\ R(t_{Task}) &= e^{-\lambda \cdot (t_{op1} + t_{op2})} = e^{-\lambda \cdot t_{Task}} \end{aligned} \quad (1)$$

Equation 1 shows the analytical way to calculate the reliability of a single task and can be derived directly by a simple Markov model consisting of two states (good and failed state) and constant failure rate λ . As mentioned before, the events of hardware faults are exponentially distributed in time. For a duplicated task the analytical formula [14] is extended to

$$R(t) = 2 \cdot e^{-\lambda \cdot t} - e^{-2 \cdot \lambda \cdot t}. \quad (2)$$

The problem with Equation 2 is the restrictions in the resolution of numbers in computer systems. In real applications, the error rate λ is a very small number ($\approx 10^{-9} \text{ h}^{-1}$). The execution time of a task is usually a small number, too. The exponent in the terms of Equation 2 goes to zero and the result of the exponentiation goes to 2 and 1. But the resolution of numbers in computer systems is limited also when using floating point numbers. With a decreasing exponent in Equation 2, the difference between both terms decreases till it is smaller than the resolution of the number system. The subtraction of both terms always leads to 1. The consequence is that an analytical method can not be applied here. In Section III-B, we present a numerical method for the evaluation of the reliability based on an n-staged continuous-time Markov model.

A. Constraints

The first constraint of the following evaluation is the assumption of only sporadic transient faults (=“soft errors”) that cause data errors during the software processing. In contrast to permanent faults, the recovery of transient faults is possible during the runtime of a task. On the other hand, faults that result in program flow errors can only be detected by a special flag technique [16] or the time redundant execution as described in [15]. Program flow errors and their detection are not the topic of this paper.

The rate of soft errors in integrated circuits exceeds up to 50.000 FIT in worst case (see [17]) and they are more probable than permanent errors. This means that we can assume a fault rate of

$$\lambda = \frac{50.000 \text{ failures}}{10^9 \text{ h}} = 0.00005 \text{ h}^{-1} \quad (3)$$

for our evaluation of task reliability. This rate is supposed to be constant and the technique of Markov processes can be used. The failure state of a task system is the condition wherein the result of a computation in the task leads to incorrect values and this is not detected by the task. Otherwise, the system is in the operating state, if the result is correct or the error can be detected.

In the course of this section, we do the comparison based on the reliability of

- a single, uncoded task (Section III-B),
- a duplicated task (Section III-C) and
- a coded task for single error detection (Section III-D).

B. Uncoded Task

An uncoded task is a simple task without any additional redundancy. It is executed only once and errors in the result of the task can not be detected. This case will be the reference for later comparison with different realizations of task processing. The reliability model of a simple, uncoded task consists of only one single component that is either working (OK) or there is a fault and the task terminates with incorrect data (NOK). Because of the previously mentioned limitations in the resolution of computer arithmetic, Equations 1 and 2 can not be applied in all cases. The idea is to model the short time of task execution with several stages in the Markov model. The basic Markov model of a non-repairable system consists of two states that are enhanced by n stages. At the end of the stages, there are two absorbing states which represent the termination of the task (Figure 1). The model of Figure 1 is described by the set of $2 \cdot (n + 1)$ differential equations:

$$\begin{aligned} P'_{1,0} &= -\lambda \cdot P_{1,0} - n\mu \cdot P_{1,0} \\ P'_{2,0} &= +\lambda \cdot P_{1,0} - n\mu \cdot P_{2,0} \\ &\dots \\ P'_{1,k} &= -\lambda \cdot P_{1,k} - n\mu \cdot P_{1,k} + n\mu \cdot P_{1,(k-1)} \\ P'_{2,k} &= +\lambda \cdot P_{1,k} + n\mu \cdot P_{2,(k-1)} - n\mu \cdot P_{2,k} \\ &\dots \\ P'_{1,n} &= +n\mu \cdot P_{1,(n-1)} \\ P'_{2,n} &= +n\mu \cdot P_{2,(n-1)} \end{aligned}$$

The solution of the system of differential equations results in the time-dependent probabilities of each state. With increasing complexity of the model, the equations can only

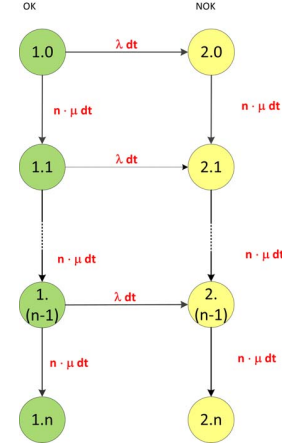


Figure 1. Enhanced Markov model that simulates the runtime of a single task with several stages. The total execution time with $t_{Task} = \frac{1}{\mu}$ is divided into n segments. The transition rate of every stage is therefore $n \cdot \mu$. The task terminates after t_{Task} either without any faults (state 1.n) or with a fault (state 2.n). This kind of task does not offer the possibility of fault detection and repair.

be easily solved numerically by means of software tools like MATLAB. The absorbing states of a Markov model represent the state of the system in infinity. In this model with two absorbing states, the task will terminate with certainty either in one or the other state, and the functions $p_{1,n}(t)$ and $p_{2,n}(t)$ describe the probability of being in one of these states. At the beginning, when the system is in state 1.0, the probability of being in one of the final states is zero. With progressing time, the probability of being in state 1.n or 2.n increases rapidly at the end of the execution (see Figure 2).

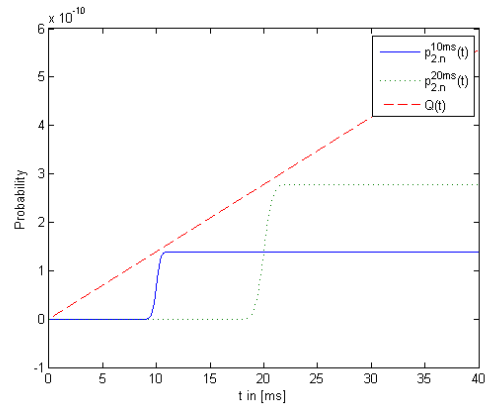


Figure 2. State 2.n represents the failure state which the system enters after termination of the task. The figure shows the probability of the failure state with two different execution times $t_1 = 10\text{ms}$ and $t_2 = 20\text{ms}$. The steady-state probability of both curves equals the probability of the failure probability calculated by $Q(t) = 1 - e^{-\lambda t}$. This means that the numerical solution by the n -staged Markov model is equivalent to the standard analytical solution.

C. Symmetric Redundant Task

The duplication of components is a common method to increase the reliability of systems. The same technique can be applied to task execution, as well. The task is executed twice either on different cores or sequentially on the same core of a micro-controller. When only transient faults are considered, the duplicated task is equivalent to a repeated execution. The second run of the task (= time redundancy) restarts with correct data like the task would do running on duplicated hardware (= space redundancy). At the end of the second task instance, the results of both are compared and a possible single fault can be detected. There are three possible outcomes for this comparison: (1) both tasks terminate without any faults and the results are equal, (2) only one task terminates with a fault and the two results differ, and (3) the results of both tasks are not correct. In the second case with different outcomes, the fault can be detected. As a result, the task is repeated and there is a low probability that after the repetition one of the tasks will terminate with a fault again. The third case is similar. If there is a different fault in each task, then the results are different, as well. But there is a small probability that both tasks will have the same faulty output and the fault is not detectable. This dangerous condition can cause a system failure in the further run of the software and defines the residue error probability of the system.

What is the probability that the two faulty results are equal? With a register width of b bits, there are b possibilities for a fault in the task result. We assume that the faults in both tasks are independent and the probability of a single fault is equal for all bits. A single fault in one result has b outcomes (one fault in one bit). With two tasks, there are b^2 different probabilities of faulty results. But only b combinations of the two results are equal. Thus, only $d = \frac{1}{b}$ -th part of the failure probability results in a dangerous undetected condition.

In our case, we have two equivalent tasks and the system is regarded as failed if both tasks have the same faulty output. The reliability model consists of two parallel nodes that can fail independently. The basic Markov model with three states is enhanced by n stages in the same way as it was done for the single uncoded task.

Description of states:

- State 1.x: No errors have occurred in either task. The results of both tasks are equal.
- State 2.x: One task has a faulty output/result. The results of the two tasks differ and the fault is detectable.
- State 3.x: Both tasks have a faulty output. An error detection is only possible if the two results are different. In case both results have the same erroneous output, the fault can not be detected.

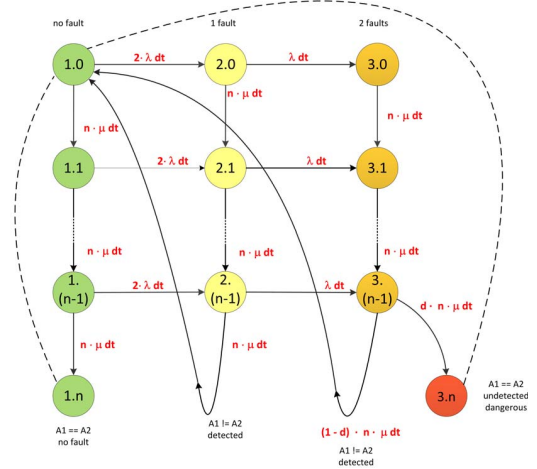


Figure 3. The basic Markov model is that of a two-component, non-repairable system with three states. It is enhanced by n stages to model the execution time of the task. The state 1.n represents the condition when the task terminates without any faults. If a faulty output is detected, then the task is repeated and the model has additional transitions to default state 1.0. There is a small probability that the erroneous output of both tasks is not detectable. In this case, the task will also terminate. This condition represents the dangerous failure state of the task system.

The absorbing states of the Markov model in Figure 3 represents the probability of the task's outcome. The solution of the set of differential equations leads to the time-dependent probabilities of both ways the task can end:

- (1) $p_1(t)$ is the probability of being in state 1.n. This state is entered when no faults have occurred during the runtime of the task. The steady-state of $p_1(t)$ represents the probability of termination in this state:

$$R(t_{Task}) = \lim_{t \rightarrow \infty} p_1(t) \quad (4)$$

- (2) $p_3(t)$ is the probability of being in state 3.n, which represents the termination of the task with undetected faults. This is the dangerous condition, because the task will terminate with a fault and it could propagate a system failure.

$$Q(t_{Task}) = \lim_{t \rightarrow \infty} p_3(t) = p_{failure} \quad (5)$$

In most software applications, a task is executed many times. These periodic tasks are restarted again after their termination. With this assumption of non-stop activity, the absorbing states in Figure 3 are obsolete and the transitions into the absorbing states move to the default state 1.0 instead. The modified model allows us to compute the steady-state probabilities $p_{1.0}, p_{1.1}, \dots, p_{3.n-1}$ of all states. Three interesting parameters can be calculated with this modified model:

1.) Mean frequency of successful execution of the task:

$$f_1 = p_{1,n-1} \cdot n \cdot \mu \quad (6)$$

2.) Mean frequency of detected unsuccessful execution:

$$\begin{aligned} f_2 &= p_{2,n-1} \cdot n \cdot \mu + p_{3,n-1} \cdot n \cdot \mu \cdot (1-d) \\ &= (p_{2,n-1} + (1-d) \cdot p_{3,n-1}) \cdot n \cdot \mu \end{aligned} \quad (7)$$

3.) Mean frequency of undetected unsuccessful (dangerous) execution:

$$f_3 = d \cdot p_{3,n-1} \cdot n \cdot \mu \quad (8)$$

The parameter f_3 is perhaps the most interesting one. This frequency represents the average occurrence of task termination with undetected faults. This is the dangerous case and the reciprocal value of f_3 represents the average duration between two failures during non-stop task execution:

$$MTBF = \frac{1}{f_3} \triangleq \frac{1}{\lambda_{DU}} \quad (9)$$

D. Diverse Redundant Task

Instead of task duplication, coded task processing uses arithmetic error codes for error detection. These diverse representations of data contain additional redundancy that can be used for checking the data validity. The pure time redundancy of the duplicated task is replaced by the more effective information redundancy. The model a) in Figure 4 shows the parallel structure of a duplicated task from the previous section in form of a reliability model. One task is executed twice as instance A and B, either on different cores (space redundancy) or sequentially on the same core (time redundancy). A subsequent comparator C verifies the results at the end of the two task instances. Because of the parallelism of the instances A and B, the probability of a single fault in one of the two tasks is doubled compared with a single task.

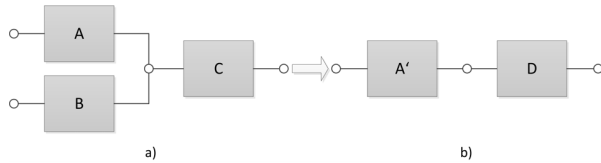


Figure 4. Change of the reliability model: The parallel structure of a duplicated task a) is replaced by the single structure of a coded task b).

In contrast to the doubled execution, the coded task is executed only once. The reliability model consists of a single component A' (see b) in Figure 4) whose probability of a single fault is half compared to the duplicated case. This means that with half execution time the fault rate of the coded task is only half compared to the duplicated case (see fault rate $2\lambda \rightarrow \lambda$ in Figure 3 and 5). But with regard to

the error detection capability, both models are equivalent provided that single error detecting codes are used. This allows us a better comparison of both techniques.

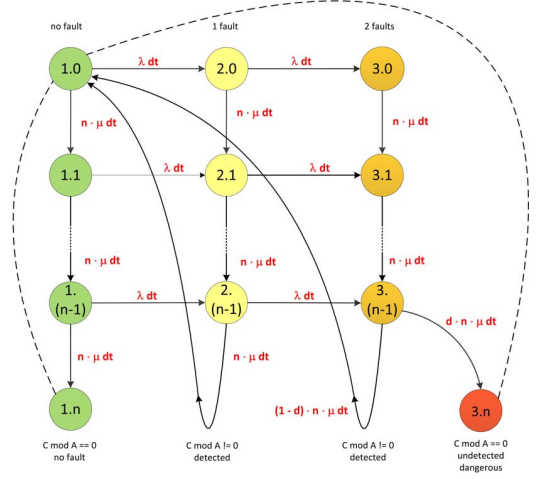


Figure 5. With the given code, all single faults are detectable. This is equivalent to a two-component, non-repairable system which detects all single faults. Because of the fact that there is only one instance of a task, the probability of a single fault is half compared to the duplicated task, whose runtime is twice as long.

Depending on the used error code, the capability of detecting errors varies enormously. We refer to [12], [11], [18], [19]. They describe a lot of codes used for detecting errors in arithmetic operations. When using an error code, there is also a residue error probability of undetection (see state 3.n in Figure 5). It is not scope of this paper to discuss this probability. Interested readers are referred to literature cited before. In this analysis, we assume the same residue error probability as for the duplicated task (for reasons of comparability).

E. Comparison

In the previous sub-sections, we described three different realizations of task execution. A single uncoded task as a reference model is now compared with a duplicated and a coded task. Using the described models in Figure 1,3 and 5, it is possible to calculate the numerical results for a) the probability of undetected failure and b) the mean time between failure (= MTBF). The comparison of these results are summarized in Table I.

The comparison does not consider the increased runtime of coded tasks with coded operations. With increasing runtime, the probability of a fault will increase [14] and of course the probability of undetected faults is higher. However, the technique with coded task is preferable, particularly with the background that error codes can provide better error detection (and correction) capabilities, instead of multiplication of the parallel structure in Figure 4 a).

	$P_{failure}$	MTBF
Simple Task:	$1.388 \cdot 10^{-10}$	$7.200 \cdot 10^9 \text{ ms}$
Symmetric Redundant Task:	$271.3 \cdot 10^{-21}$	$47.40 \cdot 10^{18} \text{ ms}$
Diverse Redundant Task:	$135.6 \cdot 10^{-21}$	$94.79 \cdot 10^{18} \text{ ms}$

Table I

COMPARISON BETWEEN DIFFERENT REALIZATIONS OF TASK PROCESSING. ($t_{task} = 10\text{ms}$, $n = 8$, $\lambda = 0.0005 \text{ h}^{-1}$, $d = \frac{1}{8}$)

IV. CONCLUSION

The reliability analysis of software systems is becoming more important for evaluation of safety-critical applications. In this paper, we presented a technique based on an enhanced Markov model for investigating the probability of undetected failures during task processing. The analytic solution for this problem is restricted by the limited resolution of number representation in computer systems, especially for tiny failure rates and short task execution times. The presented idea is to model the runtime of a task by an n-staged Markov model to compensate for the restriction mentioned above. The solution of the system of differential equations derived by this enhanced Markov model results in the time-dependent state probability which is the basis for further investigations such as, for example, the steady-state probabilities of a possible outcome of tasks. Further enhancement of the model with the assumption of non-stop execution leads to the computation of the mean time between failure (= MTBF). The comparison of three possible realizations as presented in this paper, shows that the reliability of task processing is increased by additional redundancy, especially information redundancy in form of arithmetic codes (see Table I). In contrast to the described improvements, decreased performance must be considered as an open item. The duplicated task execution requires more resources, either additional hardware or runtime. The coded task also needs more time because of the coded operations and the decoder at the end. Further investigations have to be done for the analysis, whether there is an influence where and how many synchronization points are during the task processing as described in [15].

REFERENCES

- [1] P.E. Dodd and L.W. Massengill. Basic mechanisms and modeling of single-event upset in digital microelectronics. *Nuclear Science, IEEE Transactions on*, 50(3):583 – 602, June 2003.
- [2] Olga Golubeva, Maurizia Rebaudengo, Matteo Sonza Reorda, and Massimo Violante. *Software-Implemented Hardware Fault Tolerance*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2006.
- [3] Functional safety of electrical/electronic/programmable electronic safety-related systems - part 1: General requirements, April 2010.
- [4] N. Oh, S. Mitra, and E.J. McCluskey. ED4I: Error Detection by Diverse Data and Duplicated Instructions. *IEEE Transactions On Computers*, Vol. 51, pages 180–199, 2002.
- [5] IEEE standard glossary of software engineering terminology, September 1990.
- [6] R. Billinton and R.N. Allan. *Reliability evaluation of engineering systems: concepts and techniques*. Plenum Press, 1992.
- [7] Magdi S. and Moustafa. Availability of k-out-of-n:g systems with m failure modes. *Microelectronics Reliability*, 36(3):385 – 388, 1996.
- [8] K.D. Thies. *Elementare Einführung in die Wahrscheinlichkeitsrechnung, Informationstheorie und stochastische Prozesse*. Shaker Verlag, 2010.
- [9] E. Härtter. *Wahrscheinlichkeitsrechnung, Statistik und mathematische Grundlagen: Begriffe, Definitionen und Formeln*. Vandenhoeck & Ruprecht, 1987.
- [10] V. Vais and S. Racek. Experimental evaluation of regular events occurrence in continuous-time markov models. In *Informatics, 2011*, Nov. 2011.
- [11] P. Forin. Vital coded microprocessor principles and application for various transit systems. In *IFA-GCCT*, pages 79–84, 1989.
- [12] Thammavarapu R. N. Rao. *Error coding for arithmetic processors*. Electrical science series. Academic Press, New York and London, 1974.
- [13] J. Mottok, F. Schiller, Th. Völkl, and Th. Zeitler. A Concept for a Safe Realization of a State Machine in Embedded Automotive Applications. In *26th Safecomp Conference, ISBN 978-3-540-75100-7*, pages 283–288, 2007.
- [14] M. Steindl, J. Mottok, and H. Meier. SES-based framework for fault-tolerant systems. In *Intelligent Solutions in Embedded Systems (WISES), 2010 8th Workshop on*, pages 12 –16, July 2010.
- [15] P. Raab, S. Kramer, J. Mottok, H. Meier, and S. Racek. Safe software processing by concurrent execution in a real-time operating system. In *Applied Electronics (AE), 2011 International Conference on*, pages 1 –5, Sept. 2011.
- [16] N. Oh, P.P. Shirvani, and E.J. McCluskey. Control-flow checking by software signatures. *Reliability, IEEE Transactions on*, 51(1):111–122, March 2002.
- [17] R. Baumann. Soft errors in advanced computer systems. *Design Test of Computers, IEEE*, 22(3):258 – 266, may-june 2005.
- [18] P. Ozello. The coded microprocessor certification. In *International Conference on Computer Safety, Reliability and Security*, pages 185–190. Springer Munich, 1992.
- [19] Ute Schiffel, André Schmitt, Martin Süßkraut, and Christof Fetzter. ANB- and ANBDMem-encoding: Detecting hardware errors in software. In *Computer Safety, Reliability, and Security*, volume 6351 of *Lecture Notes in Computer Science*, pages 169–182. Springer Berlin / Heidelberg, 2010.